RAG Practice Knowledge Base

A small sample PDF for building and testing a PDF question-answering chatbot.

Use this document as the uploaded PDF in your project. It contains clear facts, rules, schedules, and FAQ-style information so that you can test whether your retriever and LLM are finding the correct context.

Document version: 1.0

Prepared for: Beginner RAG chatbot practice

Important note: This is a fictional handbook created only for practice. Names, emails, dates, and policies are invented.

Section	What it helps you test
AI Learning Club	Simple factual retrieval
Weekly Schedule	Time and place questions
Project Rules	Deadline, penalty, and requirement questions
Resource Policy	Borrowing and access questions
RAG FAQ	Conceptual questions about the pipeline

1. AI Learning Club Overview

The AI Learning Club is a fictional student club designed for beginner practice. Its goal is to help students learn Python, machine learning, LangChain, embeddings, vector databases, and small generative AI projects.

The club office is open from **Monday to Friday, 10:00 AM to 6:00 PM**. The office is closed on Saturdays, Sundays, and public holidays.

The main club room is **Room B204**. Students can use Room B204 for study sessions, code practice, and project discussions when no workshop is running.

The club coordinator is **Maya Sharma**. General questions should be sent to **ai.club@example.edu**. Project submission questions should be sent to **projects.ai@example.edu**.

Membership is free for all students. New members must complete a short orientation before joining lab-based workshops.

1.1 Beginner Learning Path

- Week 1 focuses on Python basics, functions, loops, and simple file handling.

- Week 2 focuses on NumPy, pandas, and reading CSV files.

- Week 3 focuses on embeddings, semantic search, and vector databases.

- Week 4 focuses on building a simple PDF chatbot using retrieval augmented generation.

- Week 5 focuses on turning a notebook into a small Python project with a clean folder structure.

1.2 What Students Should Understand

By the end of the beginner learning path, students should be able to explain the difference between a normal chatbot and a RAG chatbot. They should also be able to describe what a loader, splitter, embedding model, vector database, retriever, prompt, LLM, and chat history do.

2. Weekly Schedule and Workshops

The weekly schedule is intentionally simple so that students can test time-based retrieval questions.

Day	Time	Session	Location
Monday	3:00 PM - 4:00 PM	Python doubt clearing	Room B204
Tuesday	4:00 PM - 5:30 PM	Python basics workshop	Room B204
Wednesday	2:00 PM - 3:30 PM	Vector database practice	Lab C110
Thursday	5:00 PM - 6:00 PM	Mini project review	Room B204
Friday	1:00 PM - 2:00 PM	Career and portfolio session	Seminar Hall A

Students must register before attending the Vector Database Practice session because Lab C110 has only **24 computers**. Registration closes every Tuesday at **8:00 PM**.

The Python Basics Workshop does not require registration. However, students are expected to bring their own laptop if possible.

The Mini Project Review session is for students who already have a project idea or a working prototype. Each student gets a maximum of **8 minutes** for feedback.

2.1 Recommended Questions to Test Your Chatbot

Try asking: What time is the Python basics workshop? Where is the vector database practice session? How many computers are available in Lab C110? When does registration close for the vector database session?

3. Mini Project Rules

The final beginner mini project is a PDF question-answering chatbot. Students may use OpenAI, Gemini, or a local Ollama model. The recommended beginner option is to start with a hosted API because it is easier to debug. After the pipeline works, students can try a local model.

The project submission deadline is **25 July 2026 at 11:59 PM**. Late submissions receive a penalty of **5 percent per day**. Submissions more than **5 days late** are not accepted unless the coordinator gives written approval.

Every project must include a README file, a requirements.txt file, and a short demo screenshot. The README should explain the project idea, setup steps, how to run the app, and examples of at least three questions asked to the chatbot.

Students may use datasets or documents from Kaggle, UCI Machine Learning Repository, Hugging Face datasets, government open-data portals, or self-created notes. Students must not upload private documents, passwords, API keys, or confidential files to GitHub.

3.1 Minimum Technical Requirements

- The app must load at least one PDF file.

- The PDF must be split into chunks before embedding.

- The chunks must be converted into embeddings.

- The embeddings must be stored in a vector database.

- The retriever must return relevant chunks for a user question.

- The LLM must answer using the retrieved context.

- The app must show or log the retrieved source chunks for debugging.

3.2 Suggested Folder Structure

A simple project can use the following structure: app.py for the interface, ingest.py for loading and indexing the PDF, rag_chain.py for the retriever and LLM chain, data/ for PDFs, vectorstore/ for saved embeddings, and requirements.txt for packages.

4. Resource and Lab Policy

The AI Learning Club provides a small number of shared resources. These rules are included so that your chatbot can answer policy-style questions.

Students may borrow a club laptop for a maximum of **2 hours**. A student ID card must be deposited while borrowing a laptop. Laptops cannot be taken outside the club area.

Students may use the GPU workstation only after booking a slot. GPU slots are limited to **90 minutes** per student per day. Long model training jobs are not allowed during beginner workshop hours.

The club provides Wi-Fi access for learning purposes only. Downloading large datasets above **2 GB** requires permission from the coordinator.

Food and open drinks are not allowed near lab computers. Water bottles must be kept on the side table, not beside the keyboard.

4.1 API Key Safety Rule

Students must store API keys in a .env file and must not push the .env file to GitHub. The .gitignore file should include .env, __pycache__/, vectorstore/, and any private PDF files.

If an API key is accidentally pushed to a public repository, the student should immediately delete the key from the provider dashboard and create a new key. Simply deleting the GitHub file is not enough because the key may remain in commit history.

5. RAG Chatbot FAQ

Question: What is the main difference between a normal chatbot and a RAG chatbot?
Answer: A normal chatbot answers mostly from the model knowledge. A RAG chatbot first retrieves relevant information from external documents and then asks the LLM to answer using that retrieved context.

Question: Why do we split a PDF into chunks?
Answer: We split a PDF because long documents are too large to send fully to the LLM every time. Chunks make retrieval easier and keep the context focused.

Question: What is an embedding?
Answer: An embedding is a numerical representation of text meaning. Similar texts have vectors that are close to each other in vector space.

Question: What does the vector database store?
Answer: The vector database stores document chunks, their embeddings, and usually metadata such as page number or source file name.

Question: What does the retriever do?
Answer: The retriever compares the user question embedding with the stored chunk embeddings and returns the most relevant chunks.

Question: What does the LLM do after retrieval?
Answer: The LLM reads the user question and the retrieved chunks, then writes a natural language answer based on that context.

Question: Where is chat history stored in a simple notebook chatbot?
Answer: In a simple notebook chatbot, chat history is often stored in a Python list such as self.chat_history. It is temporary unless saved to a file or database.

Question: Why is source document display useful?
Answer: Source document display helps debug the chatbot because it shows which chunks were used before the LLM generated the answer.

5.1 Test Questions for Retrieval

- What is the project submission deadline?

- How much is the late submission penalty?

- Where is the Wednesday vector database practice session?

- How long can a student borrow a club laptop?

- What should students do if they accidentally push an API key to GitHub?

- What does the retriever do in a RAG chatbot?

6. Simple RAG Pipeline Explanation

This section explains the same pipeline that students will implement in code.

Step 1: Load the PDF. The loader reads the PDF and turns pages into document objects. Each document object usually contains page text and metadata.

Step 2: Split the documents. The splitter breaks long page text into smaller chunks. Chunk overlap helps avoid losing meaning at chunk boundaries.

Step 3: Create embeddings. The embedding model converts each chunk into a vector. This vector represents the meaning of the text.

Step 4: Store in a vector database. The vector database stores chunks and vectors so that similar text can be searched quickly.

Step 5: Ask a question. The user asks a question in the app. In a conversational chain, chat history may be used to rewrite unclear follow-up questions into standalone questions.

Step 6: Retrieve context. The retriever searches the vector database and returns the top matching chunks.

Step 7: Generate answer. The LLM receives the question and the retrieved chunks. It writes the final answer using the retrieved context.

Step 8: Save chat history. The app saves the user question and the LLM answer so that future follow-up questions can be understood.

6.1 Important Debugging Tip

If the chatbot gives a wrong answer, first check the retrieved chunks. If the chunks are wrong, improve chunking, embeddings, or retriever settings. If the chunks are correct but the answer is wrong, improve the prompt or use a better LLM.